

ZURION Store — tâches réalisées et restantes

Synthèse des éléments réalisés parmi la checklist • Validation locale du 4 septembre 2026

STATUT : réalisé et vérifié localement

La première partie liste les tâches effectuées. La seconde récapitule les tâches restantes. Ce document ne constitue pas une certification de production.

1. Cadrage, architecture et préparation

- Stack technique, architecture applicative et organisation des dossiers documentées.
- Routes principales, entités, schéma de base et relations identifiés.
- Rôles client/admin et règles d'authentification documentés.
- Variables d'environnement, installation, seed et migrations documentés.

2. Accueil, navigation et contenu public

- Header, logo, navigation, accès compte et panier fonctionnels.
- Accueil avec hero, CTA, promotions, nouveautés, produits populaires et catégories.
- Éléments de réassurance et footer complet avec liens d'assistance et légaux.
- Pages accueil, catalogue, contact, FAQ, livraison et retours disponibles.

3. Catalogue et fiches produits

- Catalogue paginé avec recherche, catégories, filtre de prix, stock et tri.
- Suggestions de recherche et état sans résultat pris en charge.
- Fiches produit avec prix, promotion, galerie, miniatures, description, caractéristiques et stock.
- Ajout au panier, favoris, notation, avis et gestion d'un produit inexistant.

4. Panier, checkout et commandes

- Panier invité et panier utilisateur avec ajout, modification, suppression et vidage.
- Contrôle des quantités et du stock côté serveur.
- Checkout avec adresse, livraison standard/express/retrait et paiement à la livraison.
- Création de commande transactionnelle avec instantané d'adresse, total, frais et stock.
- Codes promotionnels avec remise fixe/pourcentage, validité et consommation.
- Confirmation de commande et historique client.

5. Administration, stocks et livraisons

- Back-office protégé par authentification et rôle administrateur.
- CRUD produits, catégories, coupons et livreurs.
- Gestion des prix, promotions, disponibilité, images et stock.
- Consultation des commandes et modification contrôlée de leur statut.
- Création automatique d'une livraison à l'expédition.
- Suivi, livraison et annulation des livraisons avec transitions contrôlées.

6. Sécurité, données et robustesse

- Hashage des mots de passe, sessions JWT httpOnly et invalidation après changement de mot de passe.
- Protection CSRF, Helmet/CSP, rate limiting et validation serveur.
- Contrôle d'accès aux ressources privées et aux routes admin.
- Messages d'erreur serveur contrôlés sans exposition inutile.
- SQLite et PostgreSQL pris en charge ; migration versionnée et idempotente.

7. QA, responsive et documentation

- 11 tests Node réussis sur 11.
- 4 tests Playwright réussis sur 4, en Chromium desktop et mobile.
- Benchmark HTTP local ajouté et exécuté sans erreur.
- README racine, documentation d'architecture et rapport de validation ajoutés.
- Pages mentions légales, confidentialité, cookies et CGU ajoutées.

Preuves principales : `npm test (11/11)`, `npm run test:browser (4/4)`, `npm run migrate` (réussi), PostgreSQL local validé et scénario panier → commande → stock → coupon → livraison exécuté.

8. Tâches restant à accomplir

À faire avant une mise en ligne publique

- Configurer le dépôt et le suivi GitHub dès que le compte et le remote seront disponibles.
- Déployer sur une infrastructure Internet cible et vérifier le domaine, le HTTPS, les variables secrètes et les sauvegardes.
- Remplacer les informations légales provisoires par les coordonnées définitives de l'exploitant et de l'hébergeur.
- Intégrer puis tester un prestataire de paiement réel (Mobile Money et carte) ; les paiements simulés restent interdits par défaut en production.
- Tester une base PostgreSQL hébergée avec TLS et valider la procédure de

migration/sauvegarde/restauration.

- Automatiser les parcours SSR des produits, catégories, coupons et livraisons.
- Compléter la recette responsive avec une tablette et réaliser un audit WCAG/accessibilité exhaustif.
- Calculer la couverture de code et exécuter un test de charge distribué sur l'infrastructure cible.
- Réexaminer les vulnérabilités transitives signalées par `npm audit --omit=dev` et appliquer une stratégie de mise à jour compatible.
- Effectuer une recette finale de production avec comptes, paiements, emails, observabilité et plan de reprise.

Les éléments restants sont explicitement séparés des fonctionnalités déjà validées afin de ne pas présenter le MVP local comme entièrement prêt pour la production.